LEARNING CONGESTION MAPS FROM PATH PLANNING CONFLICTS: STRUCTURED SIGNALS, LIMITED GAINS FOR DECOUPLED SCHEDULING

Anonymous authors
Paper under double-blind review

ABSTRACT

Decoupled multi-agent scheduling is attractive because it preserves modularity: tasks are assigned first and path planning is delegated to a downstream solver. Its main weakness is that the allocator is blind to congestion, so assignments that look cheap geometrically can create avoidable bottlenecks. We studied Conflict Memory Allocation (CMA), a lightweight feedback loop that aggregates planner-reported conflicts into a spatial memory map and reuses that map as a soft penalty during future assignment. The retained artifact supports only part of the original hypothesis. The learned conflict memory is clearly non-uniform: the final map is sparse, with a few persistent hotspots that align with repeated bottleneck regions. However, these structured signals translated into little overall benefit. In the available evaluation, the best logged penalty weight was 0.5, the corresponding labeled final improvement was only 0.0123, and the logged improvement-ratio values were almost always near zero with one substantial negative outlier near -0.2. The main lesson is that learning where conflicts happen is easier than turning that knowledge into a decision signal strong enough to improve a decoupled allocator.

1 INTRODUCTION

Task allocation followed by path planning remains a practical default in multi-agent systems because the pipeline is simple, modular, and scalable. The downside is equally familiar: an assignment that appears short when each agent is considered independently can funnel many agents through the same aisle, doorway, or pickup region, leaving the path planner to absorb the damage. This motivates a simple question: can the planner's own conflicts be recycled as feedback for later allocations?

Our starting hypothesis was optimistic. If path-planning conflicts reflect stable features of the environment, then a lightweight congestion prior might recover some of the benefit of coupling without paying the cost of joint optimization. This is a small representation-learning problem: repeated local events are compressed into a reusable bias for future decisions (Goodfellow et al., 2016). We therefore implemented Conflict Memory Allocation (CMA), a histogram-based memory that stores where conflicts happen and adds a soft penalty to assignments whose estimated routes traverse those locations.

The pilot logs only partially support this idea. The positive result is that conflict memory is *learnable*: the final map is highly structured, sparse, and dominated by a few repeatable hotspots. The negative result is that this structure does not reliably improve scheduling. Across the available evaluation traces, the logged improvement-ratio values are visually concentrated at zero, the artifact's best-tuned run reports only a tiny aggregate gain, and one instance shows a clear regression. For real deployments, this is a useful failure case: detecting bottlenecks is not enough if the learned signal is too weak or too poorly aligned with the optimizer that is supposed to use it. We emphasize that the retained artifact supports a cautionary empirical claim, not a universal impossibility result.

2 RELATED WORK

Our study sits between strict decoupling, where assignment and planning are solved separately, and tighter integration, where downstream planning information influences upstream decisions. CMA tries to keep the modularity of the former while borrowing a small amount of feedback from the latter through learned cost shaping. In that sense, it follows the broader machine-learning view that a useful representation is one that changes a downstream decision rule rather than merely describing the data (Goodfellow et al., 2016). The negative result here is exactly a failure of that last step: the learned conflict representation is visible and structured, but it is not strong enough to induce robust assignment changes. This framing is also why the result is relevant to an ICBINB-style workshop. The failure is not that the signal is random; the failure is that a plausible, interpretable, and apparently learnable signal still does not yield meaningful end-task improvement when inserted into a decoupled pipeline.

3 METHOD

We consider a standard decoupled pipeline. A task allocator first chooses agent-to-task assignments using a baseline cost $c_0(a, t)$, typically derived from geometric distance or single-agent travel time. A path planner then computes collision-free routes. Planner conflicts are informative because they expose where the decoupled choice created competition for space and time.

CMA summarizes those conflicts in a reusable grid memory $M \in \mathbb{R}_{\geq 0}^{H \times W}$. After solving instance k, observed conflicts are projected into a grid histogram C_k, and the memory is updated by an exponential moving average,

$$M_{k+1} = \beta M_k + (1 - \beta) C_k,$$

where β controls forgetting. CMA does not change the downstream planner. Instead, it perturbs only the assignment cost,

$$\tilde{c}(a, t) = c_0(a, t) + \lambda \sum_{v \in \hat{P}(a,t)} M(v),$$

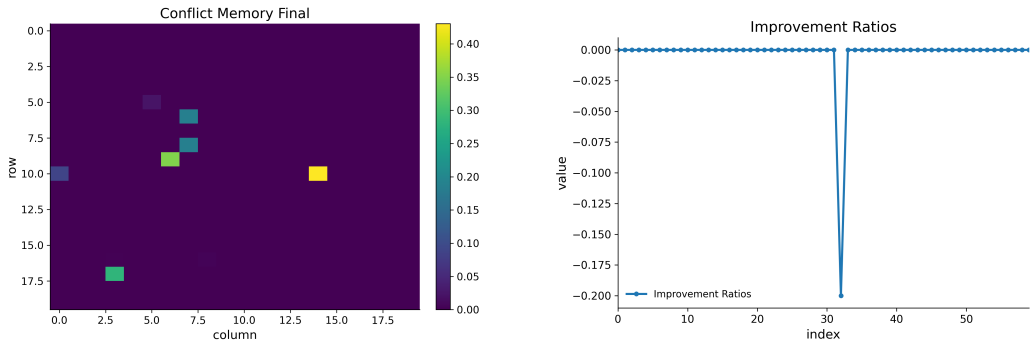
where $\hat{P}(a, t)$ is a single-agent shortest-path surrogate and λ is a penalty weight. This design is deliberately lightweight, but it also exposes the core risk: a correct memory may still fail to change the chosen assignment if the surrogate path is a poor proxy for the final joint plan or if the added penalty is small relative to variation in $c_0(a, t)$.

4 EXPERIMENTS

The retained artifact is limited, and we report it conservatively. The provided `BASELINE_SUMMARY` and `RESEARCH_SUMMARY` JSON objects are empty, so every quantitative claim below comes directly from logged visualizations rather than recomputed aggregate statistics. We therefore keep only one main-text figure: a compact diagnostic that captures the paper's central empirical contrast. Other exported images were excluded from the scientific argument because their semantics are too weakly labeled to support reliable captions; we list those files in the appendix rather than treating them as evidence.

Two exact scalars survive from the tuning logs: the best penalty weight was $\lambda = 0.5$, and the corresponding labeled final improvement was only 0.0123. Beyond those anchors, Figure 1 and Table 1 should be read diagnostically rather than statistically. The horizontal axis in Figure 1(b) is just an instance index, not training time, and the empty summary files prevent confidence intervals, seed-level aggregation, or significance tests.

The scientific value of Figure 1(a) is that it rules out a trivial failure mode. CMA did not fail because the memory remained flat. The learned map is visibly sparse and low-entropy: most cells carry negligible weight, while a small number of locations dominate the penalty. That pattern is exactly what one would expect if the environment contains recurring chokepoints. At the same time, the concentration is so sharp that the learned prior may have limited leverage. If many candidate assignments traverse the same unavoidable hotspot, then a correct memory still produces nearly identical penalties for the options the allocator is comparing, leaving the ranking mostly unchanged.



(a) Final conflict memory.

(b) Improvement-ratio values by instance index.

Figure 1: Single-run diagnostic. (a) Final 20×20 conflict-memory map. Activation is sparse and concentrated in one dominant hotspot with a few weaker secondary cells. (b) Logged improvement-ratio values by instance index. Most values are at or extremely near zero, with one harmful outlier near -0.2. Together, the panels show that CMA learned stable spatial structure but rarely converted it into better outcomes.

Table 1: Main empirical takeaways directly supported by the retained artifact.

Evidence source	Observation	Interpretation
Tuning log	Best penalty weight 0.5; labeled final improvement 0.0123	Even the best recovered setting yields only a very small aggregate gain.
Conflict-memory heatmap	Sparse map with one dominant hotspot and a few weaker cells	CMA learns persistent spatial structure rather than staying uniform.
Improvement-ratio plot	Most values lie near zero; one visible negative outlier near -0.2	Benefits are inconsistent and occasional regressions are possible.

Figure 1(b) shows the complementary failure. The downstream metric is almost zero-inflated: for most indexed instances, the improvement ratio is visually indistinguishable from zero. This suggests that CMA usually behaves as a weak perturbation rather than a decisive intervention. A useful way to interpret the negative outlier is not merely that CMA "sometimes hurts," but that historical conflict memory can redirect traffic toward a different bottleneck without reducing the true joint critical path. In other words, a map of where conflicts used to occur is not yet a counterfactual model of which reassignment will reduce final makespan on the current instance.

From a deployment perspective, the tuning result is mildly informative rather than purely null. A best weight of 0.5 indicates that the method was not disabled by a degenerate choice of $\lambda = 0$. The more interesting failure is that nonzero use of the learned memory still does not create a strong enough change in assignment rankings to matter consistently downstream.

Taken together, the figure and table point to a specific interface mismatch. CMA learns a *spatial* summary, but the downstream objective depends on *spatiotemporal* multi-agent interactions. The assignment penalty is computed on a single-agent surrogate route, whereas the planner's actual collisions depend on simultaneous occupancy, route coupling, and local timing. Under that mismatch, the memory can be accurate as a descriptive statistic and still be weak as an optimization signal. This interpretation is consistent with the tiny best logged gain: whatever useful congestion information CMA captures is too weak, too local, or too stale to robustly alter the bottleneck that determines makespan.

5 CONCLUSION

This pilot study supports only half of the original CMA story. Path-planning conflicts do contain stable, spatially structured information, and a simple histogram memory recovers that structure cleanly. But in the retained experiments, feeding this memory back into assignment produced little measurable benefit: the best logged setting barely improved the aggregate metric, the per-instance values were almost always near zero, and one outlier showed that the feedback can hurt. The practical lesson is simple: before investing in learned congestion priors for decoupled scheduling, verify not only that the signal is learnable, but also that it is aligned strongly enough with the allocator to change decisions in the right direction. A promising next step is not merely to smooth or enlarge the memory, but to redesign the feedback signal so that it better reflects timing-sensitive interactions rather than static occupancy alone.

REFERENCES

Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.

A SUPPLEMENTARY MATERIAL

A.1 FIGURE-SELECTION POLICY

We were intentionally aggressive about figure selection. Only the paired diagnostic in Figure 1 remains in the main text because it directly supports the paper's central claim: CMA learns a structured memory, yet that structure rarely translates into better outcomes. We did *not* move the other exported images into the appendix as figures, because the retained artifact does not provide enough context to caption them rigorously. Instead, we document them as unused exports below. This is a deliberate scientific choice: weakly labeled dashboards are better omitted than over-interpreted.

A.2 REPRODUCIBILITY BOUNDARIES OF THE RETAINED ARTIFACT

The machine-readable `BASELINE_SUMMARY` and `RESEARCH_SUMMARY` objects were empty. As a result, the paper intentionally avoids claims about statistical significance, exact benchmark names, or seed-aggregated performance. The only exact scalar values recoverable from the artifact were the best penalty weight 0.5 and the labeled final improvement 0.0123.

A.3 RECOVERABLE EXPERIMENTAL CONFIGURATION

Not every experimental detail survived in machine-readable form, but some configuration information can still be recovered from the artifact itself. Table 2 separates values that were directly logged from values that can only be inferred visually. We intentionally do not guess missing hyperparameters such as the memory decay factor β, benchmark identity, or seed count.

A.4 UNUSED EXPORTED FIGURE FILES

The figure directory contains several additional exports that were not used in the paper. They are documented in Table 3 for completeness, but none are shown as appendix figures because their meanings cannot be verified confidently from the retained artifact alone.

Table 2: Configuration details recoverable from the retained artifact.

Quantity	Recoverable value	Source or caveat
Conflict-memory grid size	20×20	Directly visible from heatmap axes.
Penalty weight λ	0.5	Explicitly logged as the best setting.
Final improvement	0.0123	Explicitly logged in the tuning summary.
Number of indexed instances	Dozens	Read only qualitatively from the plot extent.
Baseline/CMA per-instance traces	Available	Present as plots, not as parsed tables.
Forgetting factor β	Not recoverable	No trustworthy retained record.
Environment or benchmark name	Not recoverable	No trustworthy retained record.
Seed-level variance	Not recoverable	Empty summaries prevent reconstruction.
Exact improvement-ratio formula	Not recoverable	Only the plotted label survived.

Table 3: Unused exported figure files present in the directory.

Group	Filenames
Named summary-style exports	`benchmark_breakdown.png`, `conflict_analysis.png`, `dataoverview.png`, `dictionary_details.png`, `experiment_data.png`, `generic.png`, `main_comparison.png`, `one_dimensional_details.png`, `supplementaryresults.png`, `two_dimensional_details.png`
Supplementary-numbered exports	`supplementary_1.png`, `supplementary_2.png`, `supplementary_3.png`, `supplementary_4.png`, `supplementary_5.png`, `supplementary_7.png`, `supplementary_8.png`, `supplementary_9.png`, `supplementary_10.png`, `supplementary_11.png`